

并行程序设计 Lab3: Pthread 与 OpenMP 多线程口令猜测

姓名：刘蔚霖 学号：2412727

2026 年 5 月 28 日

摘要

本文围绕 PCFG+MD5 口令猜测程序完成基础并行化与两项进阶优化。基础部分将 `PriorityQueue::Generate` 中最后一个 segment 的 value 枚举改为 Pthread 和 OpenMP 并行，并通过预分配结果数组避免并发 `push_back`。正确性验证中，Serial、Pthread 和 OpenMP 的最终生成数量均为 10106852，Cracked 均为 358217，说明修改并没有破坏正确性。性能方面，-O2 下 OpenMP 8 线程的 Guess 阶段加速比达到 1.904，Pthread 4 线程达到 1.524。进阶部分先用细粒度 profile 定位队列维护开销，再将 `vector` 队首物理删除改为逻辑队首下标，使队首删除时间从 0.104363s 降至约 0.000018s。训练阶段并行化采用局部统计、合并、按首次出现顺序重建模型的方案；最新 `pthread_train_ordered` 数据保持 `Guesses generated=10106852`、`PopNext count=516` 和 `NewPT count=493` 一致，稳定运行的平均 Train time 从 27.3747s 降至 11.5142s。

目录

1	实验要求	3
2	实验环境	3
2.1	硬件与运行方式	3
2.2	代码组织	4
2.3	指标与可视化规划	4
3	基础要求	5
3.1	算法设计	5
3.2	串行老版本关键代码	5
3.3	Pthread 代码实现	6
3.4	OpenMP 代码实现	7

目录	2
3.5 正确性验证	8
3.6 性能测试	8
3.7 基础结果分析	11
4 进阶要求	11
4.1 OpenMP 与 Pthread 性能差异	11
4.2 编译优化对结果的影响	12
4.3 Guess 阶段细粒度 profile	12
4.4 队首删除优化	13
4.5 训练阶段并行化	15
4.6 训练并行化数据检查	17
4.7 对于并行化粒度的思考	18
5 结论	18

1 实验要求

本实验围绕 PCFG 口令猜测程序展开。程序从训练集统计口令结构和 segment value 的概率分布，根据 PCFG 模型产生候选口令，再对候选口令计算 MD5 哈希。

基础部分实现 Pthread 和 OpenMP 加速，进阶部分继续根据 profile 数据处理剩余瓶颈。基础要求是对 PCFG 猜测生成阶段进行多线程并行化。在代码中，程序在 `PriorityQueue::Generate(PT pt)` 中从优先队列取出一个 pre-terminal 后，会固定除最后一个 segment 外的所有部分，再枚举最后一个 segment 的全部 possible value。不同 value 对应的候选口令互不依赖，因此这一层枚举循环是最直接、风险最低的并行目标。进阶要求是在基础并行化之后继续分析瓶颈并做进一步优化。本文完成了两个方向：

- 对 Guess 阶段加入细粒度 profile，定位 `Generate`、`NewPTs`、`CalProb` 和队列维护的时间占比，并优化队首删除；
- 对训练阶段进行并行化，采用“线程内局部统计，主线程合并并重建模型”的思路，并用最新数据检查生成路径是否保持一致。

正确性方面，基础版本要求最终生成数量和破解结果与串行版本一致。进阶优化会改变 PCFG 生成顺序，这意味着我们必须检查 `Guesses generated`、`PopNext count` 和 `Cracked` 等指标是否仍与基准一致。

2 实验环境

2.1 硬件与运行方式

实验在 ARM 服务器节点上通过作业系统运行。计算节点的 `lscpu` 关键信息如下：

Architecture:	aarch64
CPU(s):	8
On-line CPU(s) list:	0-7
Model name:	Kunpeng-920
Thread(s) per core:	1
Core(s) per cluster:	8
NUMA node(s):	1

服务器每个作业节点提供 8 个物理核心，因此本文重点统计 1、2、4、8 线程。Pthread 早期实验过程中由于我还不知道节点的设置，提交过 16 线程的 request，但代码会将线程数上限限制为 8，所以整理数据时将其按实际 8 线程处理，不作为 16 线程结果解释。

2.2 代码组织

本实验主要代码目录如下：

- ARM_mycode/guess-serial: 串行 baseline;
- ARM_mycode/guess-pthread: 基础 Pthread 并行版本;
- ARM_mycode/guess-omp: 基础 OpenMP 并行版本;
- ARM_mycode/guess-pthread-advanced: 进阶 profile、队首删除优化和训练阶段并行化实验版本。

性能测试使用 main.cpp 编译，正确性测试使用 correctness_guess.cpp 编译。

```
# serial
g++ main.cpp train.cpp guessing.cpp md5.cpp -O2 -o main

# Pthread
g++ main.cpp train.cpp guessing.cpp md5.cpp -O2 -pthread -o main

# OpenMP
g++ main.cpp train.cpp guessing.cpp md5.cpp -O2 -fopenmp -o main
```

2.3 指标与可视化规划

本文将实验指标分成三类。第一类是正确性指标，包括 Guesses generated、Cracked、PopNext count 和 NewPT count。这些指标必须精确一致，因此主要使用表格展示。第二类是性能指标，包括 Guess time、Hash time、Train time 和 Total time，其中基础要求的加速比只针对 Guess time 计算：

$$S_{\text{guess}}(p) = \frac{T_{\text{serial,guess}}}{T_{\text{parallel,guess}}(p)}$$

第三类是瓶颈构成指标，即 profile 中各阶段占 subtotal 的比例。该类数据用于观察时间分布，因此使用组成条展示比例结构。

为了让图表与结论直接对应，本文采用以下可视化规划：

- 线程扩展性使用折线图展示，横轴为线程数，纵轴分别为 Guess time 和加速比，便于观察随线程数变化的趋势；
- Guess 阶段 profile 使用 100% 组成条展示，突出 Generate 与队列维护的占比关系；

- 队列优化使用优化前后对照图展示，重点强调 `queue_pop` 从显著开销降为近似 0，而 `queue_push` 基本不变；
- 进阶结果增加总览图，用归一化后的剩余耗时展示队列优化和训练并行化各自带来的收益；
- 正确性与原始均值保留表格，避免图形压缩造成精确数字不可读。

3 基础要求

3.1 算法设计

PCFG 生成阶段维护一个按概率排序的 PT 队列。每次 `PopNext()` 取出当前概率最高的 PT，先通过 `Generate()` 输出该 PT 对应的一批候选口令，再通过 `NewPTs()` 产生后续 PT 并插回队列。

设当前 PT 为 L8D2。当 L8 已经由 `curr_indices` 确定为某个具体 value 时，最后一个 D2 可以取模型中所有长度为 2 的数字串 value。若 D2 有 n 个候选 value，则这一步会生成 n 个候选口令。它们之间没有读写依赖，也不需要互相通信，所以可以把 $[0, n)$ 按区间划分给多个线程。

基础并行化的主要风险在结果写回。串行版本可以直接 `push_back`，多线程版本如果仍然让多个线程同时 `push_back`，会导致 `vector` 内部 `size`、`capacity` 和元素构造发生竞争。本文的 Pthread 和 OpenMP 版本都采用同一策略：主线程先一次性 `resize`，并行区域只按下标写入。

图 1 展示了本文优化在整体程序中的位置。基础要求只改动 `Generate` 内部的 value 枚举；进阶一针对 `PopNext` 中的队列维护；进阶二则尝试把训练阶段从单线程 `parse` 改为局部统计后合并。



图 1: PCFG+MD5 程序流程与本文优化位置

3.2 串行老版本关键代码

串行 baseline 的主要逻辑是顺序遍历最后一个 segment 的所有 value。若 PT 只有一个 segment，则直接输出该 segment 的全部 value；若 PT 有多个 segment，则先拼出前缀，再追加最后一个 segment 的 value。简化后的关键代码如下：

```
for (int i = 0; i < value_count; i += 1)
{
    guesses.emplace_back(prefix + values[i]);
    total_guesses += 1;
}
```

串行版本的优点是逻辑直接，生成顺序完全由循环顺序决定；缺点是最后一个 segment 的枚举无法利用多核，并且每次 `emplace_back` 都在单线程中完成。

3.3 Pthread 代码实现

Pthread 版本定义线程任务结构体，记录线程负责的区间、输出数组起点、前缀和 value 列表：

```
struct GenerateTask
{
    int begin;
    int end;
    size_t base;
    const string *prefix;
    const vector<string> *values;
    vector<string> *output;
};
```

与串行版本相比，Pthread 版本最关键的变化是预分配输出空间：

```
const size_t base = output.size();
output.resize(base + value_count);
```

然后主线程按照线程数划分区间并创建线程：

```
int block = (value_count + thread_count - 1) / thread_count;
for (int tid = 0; tid < thread_count; tid += 1)
{
    int begin = tid * block;
    int end = begin + block;
    if (end > value_count)
    {
        end = value_count;
    }

    tasks[tid] = GenerateTask{begin, end, base, &prefix, &values, &output};
}
```

```

    pthread_create(&threads[tid], nullptr, GenerateWorker, &tasks[tid]);
}

for (int tid = 0; tid < thread_count; tid += 1)
{
    pthread_join(threads[tid], nullptr);
}

```

线程函数只写入自己的下标范围：

```

void *GenerateWorker(void *arg)
{
    GenerateTask *task = static_cast<GenerateTask *>(arg);
    const string &prefix = *(task->prefix);
    const vector<string> &values = *(task->values);
    vector<string> &output = *(task->output);

    for (int i = task->begin; i < task->end; i += 1)
    {
        output[task->base + i] = prefix + values[i];
    }
    return nullptr;
}

```

这种实现不需要对每次写入加锁，因为不同线程写入的是不同元素。计数器 `total_guesses` 也不在 worker 内部更新，而是在并行循环结束后由主线程统一执行：

```
total_guesses += value_count;
```

此外，代码通过 `PCFG_THREADS` 控制线程数，并将线程数限制在 8 以内。对于 value 数量很少的情况，直接走串行 worker，避免反复创建线程的开销大于实际计算量。

3.4 OpenMP 代码实现

OpenMP 版本与 Pthread 版本采用相同的数据安全策略，即预分配后按下标写入。不同之处在于 OpenMP 不需要显式创建线程、维护 `pthread_t` 数组和任务结构体。核心代码如下：

```

const size_t base = output.size();
output.resize(base + value_count);

#pragma omp parallel for schedule(dynamic, kOpenMpChunk) \
    if (value_count > kOpenMpChunk)

```

```

for (int i = 0; i < value_count; i += 1)
{
    output[base + i] = prefix + values[i];
}

```

这里使用 `schedule(dynamic, 256)`。原因是不同 `value` 的字符串长度、拷贝和拼接开销可能不同，动态调度可以让线程在完成当前块后继续领取新块，提高负载均衡。对于小规模循环，`if (value_count > kOpenMpChunk)` 会使其保持串行执行，避免并行调度开销。

3.5 正确性验证

正确性测试使用 `correctness_guess.cpp`。串行、Pthread 和 OpenMP 基础版本最终生成数量和破解数量一致，如表 1 所示。

表 1: 基础版本正确性测试结果

版本	Guesses generated	Cracked
Serial	10106852	358217
Pthread	10106852	358217
OpenMP	10106852	358217

`Guesses generated=10106852` 表示在当前 `main.cpp` 的生成上限逻辑下，程序超过 1000 万阈值时实际累计生成的候选口令数量。由于程序按批次生成并在超过阈值后停止，所以最终值会略大于 1000 万。三个版本该值一致，并且 `Cracked` 一致，说明基础并行化没有破坏候选集合。

3.6 性能测试

每个有效配置运行 5 次并取平均。本文统计 `Guess time`、`Hash time` 和 `Total time=Guess time+Hash time`。基础加速比使用同一优化等级下的串行 `Guess time` 作为基准。

表 2: 串行 baseline 平均性能

优化等级	线程数	运行次数	Guess(s)	Hash(s)	Total(s)
O0	1	5	7.950330	10.393220	18.343550
O1	1	5	0.603787	1.125268	1.729055
O2	1	5	0.672727	1.211160	1.883887

表 3: Pthread 平均性能与 Guess 阶段加速比

优化等级	线程数	运行次数	Guess(s)	Hash(s)	Total(s)	Speedup
O0	1	5	7.880964	10.427300	18.308264	1.009
O0	2	5	7.908356	10.399320	18.307676	1.005
O0	4	5	7.660298	10.364140	18.024438	1.038
O0	8	10	7.589797	10.271660	17.861457	1.048
O1	1	5	0.601631	1.211304	1.812935	1.004
O1	2	5	0.543497	1.201696	1.745193	1.111
O1	4	5	0.514514	1.274350	1.788864	1.174
O1	8	10	0.596731	1.205748	1.802479	1.012
O2	1	5	0.517455	1.060358	1.577813	1.300
O2	2	5	0.514156	1.110664	1.624820	1.308
O2	4	5	0.441562	1.094812	1.536374	1.524
O2	8	10	0.461894	1.086821	1.548715	1.456

表 4: OpenMP 平均性能与 Guess 阶段加速比

优化等级	线程数	运行次数	Guess(s)	Hash(s)	Total(s)	Speedup
O0	1	5	7.739280	10.382520	18.121800	1.027
O0	2	5	7.614320	10.480480	18.094800	1.044
O0	4	5	7.471366	10.358300	17.829666	1.064
O0	8	5	7.439676	10.379800	17.819476	1.069
O1	1	5	0.564957	1.106756	1.671713	1.069
O1	2	5	0.478263	1.171466	1.649729	1.262
O1	4	5	0.433422	1.195314	1.628736	1.393
O1	8	5	0.389093	1.156178	1.545271	1.552
O2	1	5	0.557181	1.111946	1.669127	1.207
O2	2	5	0.461398	1.109958	1.571356	1.458
O2	4	5	0.388122	1.074916	1.463038	1.733
O2	8	5	0.353344	1.062746	1.416090	1.904

表 5 进一步抽取 -O2 下的关键结果。基础版本最终比较主要看这一组数据，因为 -O2 是实际提交和后续进阶优化使用的主要编译等级。

表 5: O2 优化等级下基础版本核心对比

版本	线程数	Guess(s)	Hash(s)	Total(s)	Speedup
Serial	1	0.672727	1.211160	1.883887	1.000
Pthread	1	0.517455	1.060358	1.577813	1.300
Pthread	2	0.514156	1.110664	1.624820	1.308
Pthread	4	0.441562	1.094812	1.536374	1.524
Pthread	8	0.461894	1.086821	1.548715	1.456
OpenMP	1	0.557181	1.111946	1.669127	1.207
OpenMP	2	0.461398	1.109958	1.571356	1.458
OpenMP	4	0.388122	1.074916	1.463038	1.733
OpenMP	8	0.353344	1.062746	1.416090	1.904

为了更直观比较 -O2 下线程数对 Guess 阶段的影响，图 2 给出了 Pthread 与 OpenMP 的 Guess time 变化趋势。

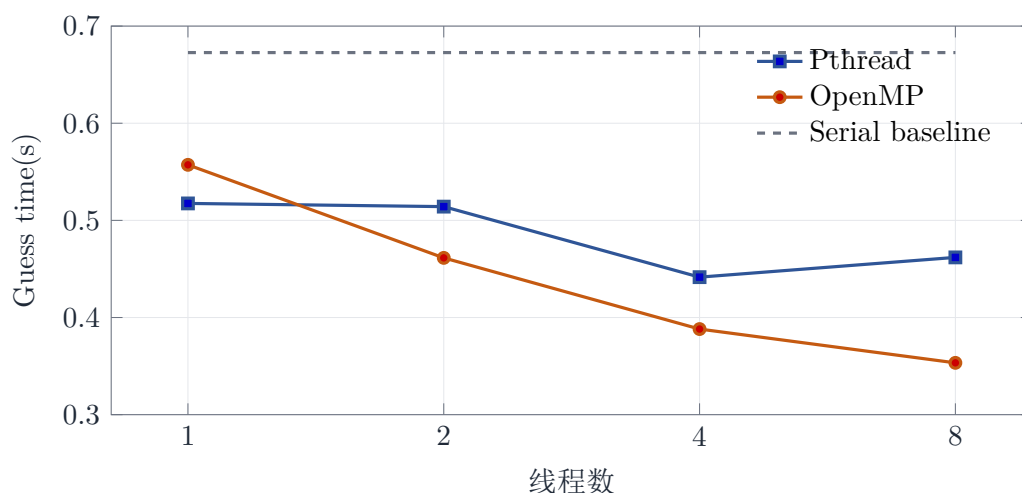


图 2: O2 下 Guess time 随线程数变化

图 3 展示了同一组数据换算后的 Guess 阶段加速比。OpenMP 的曲线随线程数增加更加稳定，而 Pthread 在 8 线程时相对 4 线程有轻微回落。这个回落也许是因为服务器波动，也有可能原因是基础版本只并行了最后一个 segment 的枚举循环，线程创建与同步、静态划分带来的等待、字符串生成的内存开销以及串行队列维护共同限制了继续扩展。

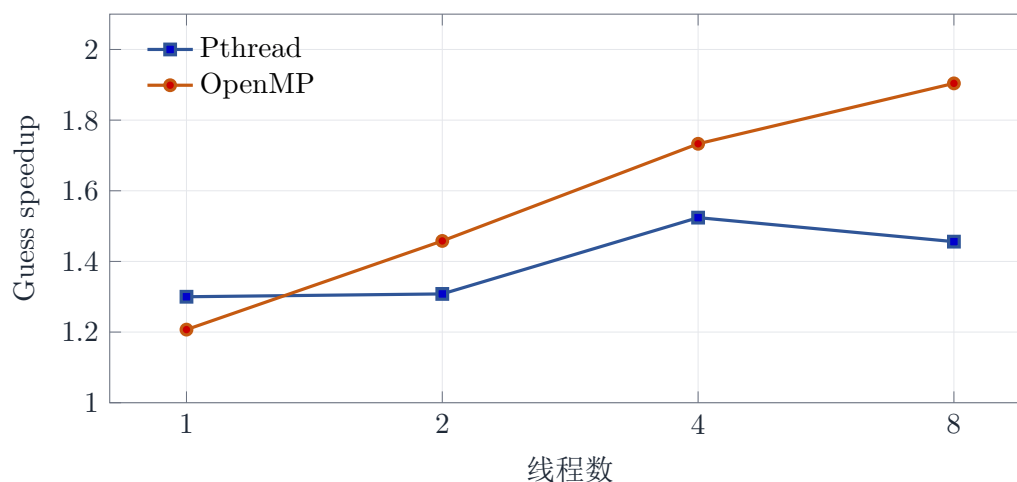


图 3: O2 下 Guess 阶段加速比

3.7 基础结果分析

从表格和图形可以看到，OpenMP 在 -O2/8 线程下效果最好，Guess time 从串行 0.672727s 降到 0.353344s，加速比为 1.904。Pthread 在 -O2/4 线程下达到 1.524 倍，8 线程时略低于 4 线程。

这个结果与代码结构相符。基础并行化只覆盖最后一个 segment 的枚举循环，训练阶段、队列维护和哈希阶段仍保持原来的执行方式；Pthread 版本在较大枚举任务上显式创建线程，再通过 pthread_join 汇合，线程创建和同步会带来固定成本；候选口令生成还包含大量字符串拼接和内存写入，线程数增加后，内存带宽和分配开销也会压缩收益。

OpenMP 版本代码更短，调度策略由运行时负责；在本实验的循环并行场景中，schedule(dynamic, 256) 的效果更稳定。Pthread 版本控制更细，但需要手动处理任务划分和线程生命周期，工程复杂度更高。

4 进阶要求

4.1 OpenMP 与 Pthread 性能差异

Pthread 和 OpenMP 选择了同一处并行区域，即最后一个 segment 的 value 枚举，但运行结果并不完全相同。Pthread 在 -O2 下 4 线程最好，OpenMP 则在 8 线程下持续提升。这个差异主要来自调度方式和线程管理方式。

Pthread 版本采用静态块划分：

```
int block = (value_count + thread_count - 1) / thread_count;
int begin = tid * block;
int end = min(begin + block, value_count);
```

这种划分实现简单，但默认假设每个 value 的处理成本接近。实际生成时，字符串长度、前缀长度、拷贝成本和缓存行为都会造成细微差异。OpenMP 使用动态调度：

```
#pragma omp parallel for schedule(dynamic, kOpenMpChunk)
```

线程完成一个 chunk 后可以继续领取新 chunk，因此负载均衡更好。另外，OpenMP 运行时通常会维护线程池，避免频繁创建和销毁线程；而当前 Pthread 实现为了保持代码简单，每次并行填充都显式 pthread_create 和 pthread_join，固定开销更明显。

4.2 编译优化对结果的影响

串行基线中 -O0 的 Guess time 为 7.950330s，而 -O1 和 -O2 分别降至 0.603787s 和 0.672727s，说明编译器优化对该程序影响很大。主要原因是 PCFG 生成阶段包含大量小函数调用、循环、字符串对象构造和容器访问，编译器优化可以减少冗余临时对象、改进循环代码并内联部分函数。

本次数据中 -O1 的串行 Guess time 略快于 -O2。对于包含 STL 容器、字符串和较复杂控制流的程序，-O2 可能改变代码布局或内联策略，使某些局部路径不如 -O1。因此本文所有加速比都只在同一优化等级内部计算，不跨优化等级混合比较。

4.3 Guess 阶段细粒度 profile

基础并行化之后，程序仍然要维护 PT 优先队列。为了定位剩余瓶颈，本文在 guess-pthread-advanced 中对 PriorityQueue::PopNext() 加入细粒度计时。计时项包括 Generate、NewPTs、CalProb、队列插入和队首删除。

```
auto generate_begin = steady_clock::now();
Generate(current);
auto generate_end = steady_clock::now();
profile_generate_time += SecondsSince(generate_begin, generate_end);

auto newpt_begin = steady_clock::now();
vector<PT> new_pts = current.NewPTs();
auto newpt_end = steady_clock::now();
profile_newpt_time += SecondsSince(newpt_begin, newpt_end);
```

队列维护部分也单独计时：

```
auto insert_begin = steady_clock::now();
/* linear scan and insert new PT */
auto insert_end = steady_clock::now();
profile_queue_push_time += SecondsSince(insert_begin, insert_end);
```

```
auto erase_begin = steady_clock::now();
priority.erase(priority.begin());
auto erase_end = steady_clock::now();
profile_queue_pop_time += SecondsSince(erase_begin, erase_end);
```

在 -O2、Pthread 8 线程下，稳定 3 次运行的平均结果如表 6 所示。

表 6: Pthread 8 线程下 Guess 阶段细粒度计时（O2，3 次平均）

计时项	平均时间 (s)	占 profile subtotal 比例
Generate 函数	0.239560	53.78%
NewPTs	0.000402	0.09%
CalProb	0.000391	0.09%
队列插入	0.100722	22.61%
队首删除 erase(begin)	0.104363	23.43%
profile subtotal	0.445438	100.00%

图 4 展示了 profile subtotal 的组成。基础并行化后的 **Generate** 仍然是最大部分，队列插入和队首删除合计达到 46.04%，已经接近总时间的一半。后续优化如果仍只关注最后一个 segment 的枚举，收益会受到队列维护开销限制。

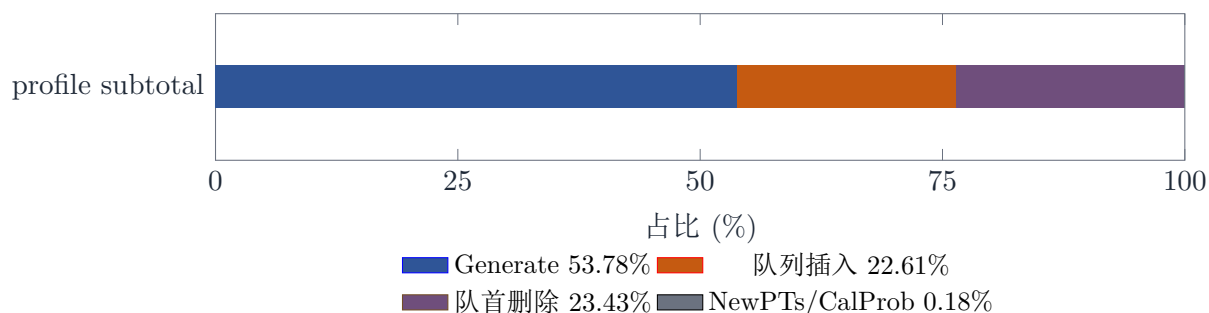


图 4: Guess 阶段 profile subtotal 组成

4.4 队首删除优化

原始版本使用 `vector<PT>` 保存按概率排序的队列。插入新 PT 时线性扫描位置，队首删除时执行：

```
priority.erase(priority.begin());
```

`erase(begin)` 会搬移后续所有元素，时间复杂度为 $O(n)$ 。直接替换为 `std::priority_queue` 虽然可以降低插入和删除复杂度，但它会改变相同概率

PT 的处理顺序，从而可能影响前 1000 万候选口令的生成路径。为了保持可比性，本文采用更保守的逻辑队首优化：保留原来的 `vector` 和插入规则，只用下标跳过已经出队的前缀。

```
vector<PT> priority;
size_t priority_head = 0;

bool PriorityQueue::Empty() const
{
    return priority_head >= priority.size();
}
```

`PopNext()` 中取队首、插入和出队分别改为：

```
PT current = priority[priority_head];

auto begin = priority.begin() + priority_head;
for (auto iter = begin; iter != priority.end(); iter++)
{
    /* keep original linear-scan insertion rule */
}

priority_head += 1;
```

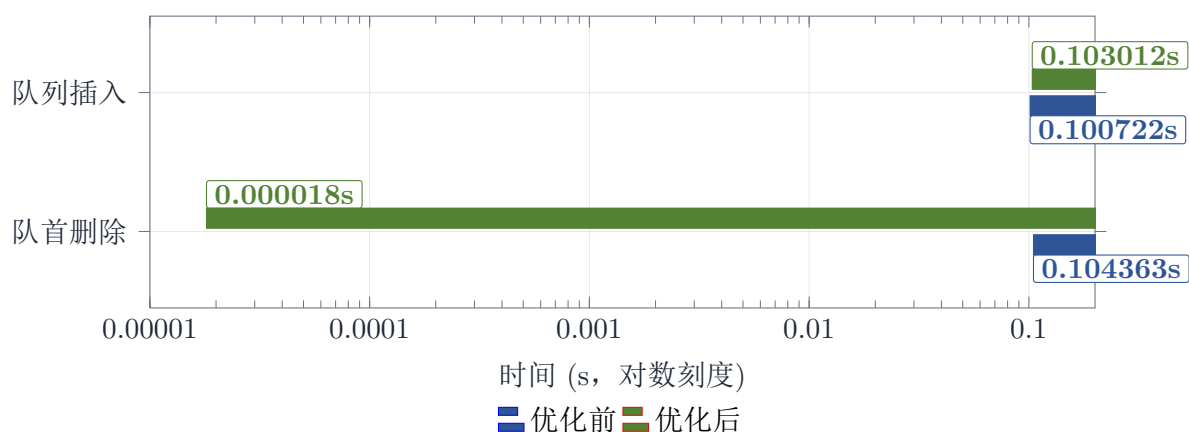
优化前后计时对比如表 7 所示。

表 7: 队首删除优化前后细粒度计时对比 (O2, Pthread 8 线程)

指标	优化前 (s)	优化后 (s)	变化
Guess time	0.483707	0.436995	加速 1.107 倍
Generate 函数	0.239560	0.282430	受运行波动影响
NewPTs	0.000402	0.000446	基本不变
CalProb	0.000391	0.000423	基本不变
队列插入/扫描	0.100722	0.103012	基本不变
队首删除	0.104363	0.000018	基本消除
profile subtotal	0.445438	0.386330	加速 1.153 倍

图 5 将队列插入和队首删除两个项目单独画出。优化后队列插入基本不变，这是预期内的结果，因为本文没有改变线性扫描插入；队首删除几乎降为 0，说明 `erase(begin)` 的搬移成本被有效消除。

优化后 5 次运行的最终生成数量均为 10106852，`PopNext count` 均为 516，`NewPT`



读图说明：横轴使用对数刻度，用来同时展示 $10^{-5}s$ 与 $10^{-1}s$ 两个量级的数据，因此条形长度不应按线性比例直接比较，结论以图中标注数值为准。队列插入优化前后分别为 0.100722s 和 0.103012s，基本保持不变；队首删除从 0.104363s 降到约 0.000018s，说明逻辑队首下标消除了 `erase(begin)` 的搬移开销。

图 5: 队列维护优化前后对比

`count` 和 `queue push count` 均为 493，与优化前一致。因此该优化没有改变可观测生成路径。

4.5 训练阶段并行化

训练阶段的串行代码一边读取训练集，一边调用 `parse(pw)` 更新模型。核心结构如下：

```
while (train_set >> pw)
{
    lines += 1;
    if (lines % 10000 == 0 && lines > 3000000)
    {
        break;
    }
    parse(pw);
}
```

直接把 `parse(pw)` 放入多个线程并对共享模型加锁并不合适。原因是 `parse` 内部会频繁更新 `preterminals`、`letters`、`digits`、`symbols` 以及对应的频数字典。如果每次更新都加锁，锁竞争会非常严重；如果只加一个大锁，整个训练过程又会退化为串行。

因此训练阶段采用局部统计思路。每个线程处理一段口令，维护自己的 `TrainStats`：

```
struct TrainStats
{
    int total_preterm = 0;
```

```
unordered_map<SegmentKey, SegmentStat, SegmentKeyHash> segments;
unordered_map<string, PTStat> preterminals;
};
```

worker 只写线程私有统计结构:

```
void *TrainWorker(void *arg)
{
    TrainTask *task = static_cast<TrainTask *>(arg);
    const vector<string> &passwords = *(task->passwords);
    TrainStats &stats = *(task->stats);

    for (size_t i = task->begin; i < task->end; i += 1)
    {
        ParsePasswordToStats(stats, passwords[i], i);
    }
    return nullptr;
}
```

所有线程结束后，主线程将局部统计量合并:

```
TrainStats merged_stats;
for (int tid = 0; tid < thread_count; tid += 1)
{
    MergeStats(merged_stats, local_stats[tid]);
}
BuildModelFromStats(*this, merged_stats);
```

训练阶段还有一个生成阶段没有遇到的问题：频数可加不代表生成路径等价。早期无序合并版本虽然降低了训练耗时，但会改变相同概率项的相对顺序，最终生成数量和 PT 迭代次数都偏离基准。为解决这个问题，`pthread_train_ordered` 不只记录 count，还记录 PT、segment 和 value 在原始输入流中的首次出现位置 `first_order`，合并后按该顺序重建模型：

```
sort(ordered_segments.begin(), ordered_segments.end(),
    [](const SegmentStat *a, const SegmentStat *b)
    {
        return a->first_order < b->first_order;
    });

sort(ordered_pts.begin(), ordered_pts.end(),
    [](const PTStat *a, const PTStat *b)
```

```
{
    return a->first_order < b->first_order;
});
```

4.6 训练并行化数据检查

最新 `pthread_train_ordered` 数据已经包含一批 -O2/8 线程运行。正文采用其中 4 次稳定运行计算均值；第 5 次运行仍保持正确性，但 `Guess time` 跳升到 1.846320s，属于服务器负载波动样本，不纳入性能均值。数据目录中还保留一次较早的单次预跑和误跑的 -O0 批次，它们只用于检查脚本输出，不参与本文结论。

表 8: 训练阶段并行化数据检查 (O2, Pthread 8 线程)

版本	运行次数	Guesses	PopNext	NewPT	Guess(s)	Train(s)
队首删除优化 <code>pthread_head</code>	5	10106852	516	493	0.436995	27.3747
训练并行 <code>train_ordered</code> 稳定数据	4	10106852	516	493	0.475252	11.5142
训练并行 <code>train_ordered</code> run5	1	10106852	516	493	1.846320	13.0750

表 8 中, `train_ordered` 稳定数据的 `Guesses generated`、`PopNext count` 和 `NewPT count` 均与队首删除优化版本一致，说明训练并行化没有改变可观测生成路径。训练阶段耗时从 27.3747s 降到 11.5142s，加速约 2.38 倍，耗时下降约 57.94%。

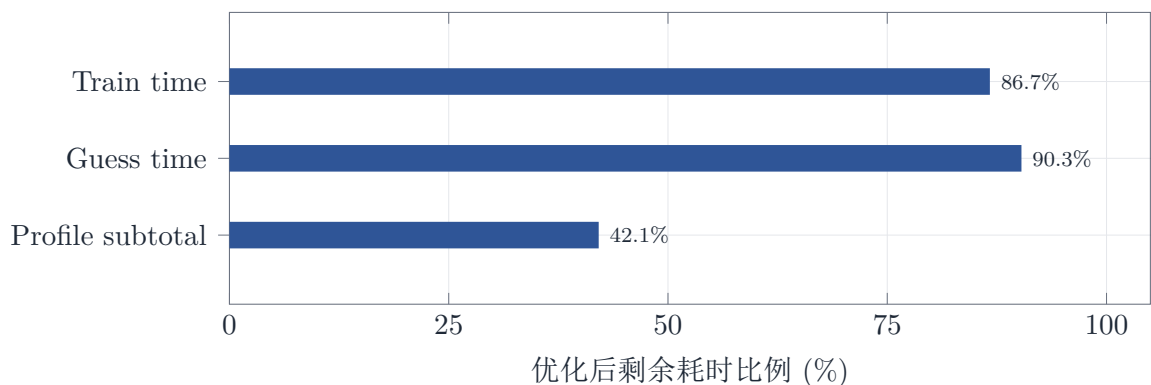


图 6: 进阶优化效果总览

图 6 采用各自优化前的耗时作为 100%。队首删除优化后, `profile subtotal` 保留 86.7%，整体 `Guess time` 保留 90.3%；训练并行化后, `Train time` 保留 42.1%。这张图比较每项优化对应的下降幅度。

4.7 对于并行化粒度的思考

本实验中最稳定的基础优化来自最后一个 segment 枚举，因为该循环的迭代之间完全独立，且结果集合容易保持一致。队首删除优化属于串行数据结构调整，但它来自 profile 指出的真实瓶颈，且不改变程序语义，因此收益可靠。

训练阶段并行化对正确性提出了更高要求。PCFG 猜测会持续从概率队列中取出 PT，并只生成前若干批候选口令；相同概率下的相对顺序一旦变化，前 1000 万个 guess 就可能不同。因此，训练并行化必须恢复模型构造时依赖的首次出现顺序。

5 结论

本文完成了 PCFG 口令猜测程序的基础并行化和两项进阶优化。基础部分中，Pthread 和 OpenMP 都对 `PriorityQueue::Generate` 中最后一个 segment 的 value 枚举进行并行化，并通过预分配结果数组、按下标写入的方式避免并发 `push_back`。正确性测试显示，Serial、Pthread 和 OpenMP 的最终生成数量均为 10106852，Cracked 均为 358217。

性能方面，在 -O2 下，OpenMP 8 线程的 `Guess time` 加速比达到 1.904，Pthread 4 线程达到 1.524。OpenMP 在本实验中扩展性更好，主要得益于更低的代码层管理成本和动态调度策略；Pthread 版本控制更直接，但线程创建、同步和静态划分带来额外开销。

进阶部分中，细粒度 profile 表明队列插入和队首删除合计约占 profile subtotal 的 46.04%。将 `vector` 队首物理删除改为逻辑队首下标后，队首删除时间从 0.104363s 降至约 0.000018s，profile subtotal 从 0.445438s 降至 0.386330s，且生成数量和 PT 迭代次数保持一致。

训练阶段并行化实现为“局部统计、合并、按首次出现顺序重建”的版本。最新 `pthread_train_ordered` 稳定数据中，最终生成数量仍为 10106852，`PopNext count` 为 516，`NewPT count` 为 493；`Train time` 从 27.3747s 降到 11.5142s，加速约 2.38 倍。